



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

*Визуализация движения автомобиля в
трехмерной городской среде с
использованием алгоритма поиска пути*

Студент

ИУ7-52Б

(группа)

(подпись, дата)

Г. А. Доколин

(И.О. Фамилия)

Руководитель курсовой
работы

К. А. Кивва

(подпись, дата)

(И.О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

1	Аналитическая часть	5
1.1	Формализация объектов синтезируемой сцены	5
1.2	Анализ алгоритмов удаления невидимых поверхностей	6
1.2.1	Алгоритм обратной трассировки лучей	6
1.2.2	Алгоритм Робертса	7
1.2.3	Алгоритм построчного сканирования с использованием буфера глубины	7
1.3	Анализ методов закрашивания	7
1.3.1	Простое закрашивание	7
1.3.2	Закрашивание Гуро и Фонга	8
1.4	Построение теней	8
1.4.1	Проекционные тени	8
1.4.2	Теневые объемы	8
1.4.3	Метод карт теней	9
1.5	Трёхмерные преобразования	9
1.6	Организация движения автомобиля и поиск пути	9
1.6.1	Анализ алгоритма Дейкстры	9
1.6.2	Анализ алгоритма A* (A-star)	10
1.6.3	Анализ волнового алгоритма	10
1.6.4	Выбор алгоритма поиска пути	10
1.7	Текстурирование	10
1.8	Выводы по аналитической части	11
2	Конструкторская часть	12
2.1	Требования к программному обеспечению	12
2.2	Описание структур данных	12
2.3	Общий алгоритм построения изображения	13
2.4	Аффинные преобразования	14
2.5	Приведение к пространству камеры	14
2.6	Перспективная проекция	14
2.7	Алгоритм построчного сканирования с Z-буфером	15

2.8	Расчет освещения и теней	15
2.9	Организация поиска пути	16
2.10	Вывод	16

Введение

Современные методы компьютерной графики позволяют создавать трёхмерные сцены, отражающие объекты и процессы реального мира. Одной из практических задач является визуализация движения транспортных средств в городской среде, где важно корректно отображать геометрию, освещение и взаимодействие с элементами дороги и препятствиями.

Разработка подобных систем требует продуманной организации сцены и применения алгоритмов, обеспечивающих удаление невидимых поверхностей, расчёт освещения, текстурирование, построение теней и реализацию движения объектов. При этом необходимо поддерживать достаточную скорость работы и удобство восприятия визуализации.

Целью курсовой работы является разработка программного обеспечения для трёхмерного моделирования движения автомобиля в городской среде.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) описать структуру трёхмерной сцены, включающую дороги, здания и элементы городской инфраструктуры;
- 2) выбрать и реализовать алгоритмы удаления невидимых поверхностей, закрашивания и построения теней;
- 3) обеспечить поддержку текстурирования и освещения от бесконечно удалённых источников света;
- 4) разработать систему управления виртуальной камерой;
- 5) реализовать поиск кратчайшего пути по дорожной сети и анимацию движения автомобиля от начальной до конечной точки;
- 6) объединить все компоненты в единую программную систему для визуализации движения в реальном времени.

Разрабатываемое приложение должно обеспечивать визуально достоверное отображение городской сцены с трёхмерными моделями зданий и дорог, поддержку текстур, реалистичное освещение и анимацию движения автомобиля по кратчайшему маршруту.

1 Аналитическая часть

В данном разделе проводится глубокий анализ существующих алгоритмов, математических моделей и программных технологий, применяемых для решения комплексной задачи трёхмерного моделирования движения автомобиля в условиях городской среды. Основной целью аналитической части является формирование чётких требований к разрабатываемому программному обеспечению, а также обоснованный выбор оптимальных методов визуализации, удаления невидимых поверхностей, расчета освещенности, построения теней и алгоритмической организации движения объекта по кратчайшему маршруту. На основании проведённого исследования формулируются критерии эффективности и технические ограничения, которые будут определять дальнейший процесс проектирования и разработки системы.

1.1 Формализация объектов синтезируемой сцены

Трёхмерная сцена представляет собой модель участка городской территории, включающую в себя здания, дорожную сеть и прочие элементы окружающей среды. Все геометрические построения и расчеты производятся в единой декартовой системе координат. Для корректной визуализации и взаимодействия объектов необходимо определить состав элементов сцены и способ их математической формализации в виде структур данных. В рамках поставленной задачи выделяются следующие ключевые типы объектов:

— **Дороги.** Данный объект представляет собой совокупность полигонов, образующих плоскость, по которой осуществляется движение. Для целей логического моделирования движения автомобиля дорожная сеть дополнительно описывается двумя двумерными массивами: дискретной матрицей проходимости, определяющей статические препятствия и свободные участки, и матрицей разрешённых направлений, которая задает правила движения на каждом конкретном участке. Геометрически дорога описывается набором вершин и граней с наложенными текстурами, имитирующими дорожное покрытие.

— **Здания.** Данные объекты моделируются как набор простейших геометрических примитивов, таких как параллелепипеды и призмы различной конфигурации. Каждое здание описывается через полигональную сетку, состоящую из списков вершин в трехмерном пространстве и списков граней, соединяющих эти вершины. Для повышения визуальной реалистичности используется текстурирование фасадов изображениями окон, дверей и материалов стен.

— **Автомобиль.** Является ключевым динамическим объектом сцены. Его геометрическая модель формируется из сложного набора вершин, рёбер и граней, образующих кузов и колеса. Положение автомобиля в пространстве однозначно задаётся вектором координат его геометрического центра и матрицей поворота, определяющей текущую ориентацию корпуса относительно осей координат. Модель поддерживает аффинные

преобразования в реальном времени для имитации езды.

— **Источник света.** Описывается вектором направления световых лучей и интенсивностью излучения. В данной задаче используется модель удалённого источника света, имитирующего солнечное освещение. Особенностью данной модели является допущение, при котором все световые лучи считаются параллельными друг другу, что упрощает расчеты теней и освещенности для всех объектов сцены.

— **Камера.** Представляет собой виртуальный наблюдатель, через которого пользователь воспринимает трехмерную сцену. Её параметры включают положение в мировых координатах, точку направления взгляда и вектор ориентации верха камеры. Все объекты сцены визуализируются путем перевода их координат в систему отсчета камеры с последующей перспективной проекцией на плоскость экрана.

Для хранения геометрической информации об объектах сцены используется полигональное представление. При таком подходе каждая поверхность описывается множеством вершин с уникальными координатами и списком граней, которые ссылаются на индексы этих вершин. Данный формат является универсальным стандартом в компьютерной графике и позволяет эффективно применять алгоритмы растеризации и трансформации.

1.2 Анализ алгоритмов удаления невидимых поверхностей

Одной из фундаментальных задач трёхмерной визуализации является устранение невидимых поверхностей. Суть задачи заключается в определении того, какие части объектов перекрываются другими объектами с точки зрения наблюдателя и, следовательно, не должны выводиться на экран монитора.

1.2.1 Алгоритм обратной трассировки лучей

Суть метода заключается в физически корректном моделировании прохождения лучей света от наблюдателя к объектам сцены. Для каждого пикселя экрана строится виртуальный луч, который пересекает трехмерное пространство сцены. Далее находится ближайшая к наблюдателю точка пересечения луча с каким-либо объектом, и цвет этой точки присваивается пикселю. **Преимущества:** Метод обеспечивает высокую физическую точность изображения, позволяет корректно моделировать сложные оптические эффекты, такие как зеркальные отражения, преломления в прозрачных средах и мягкие тени. **Недостатки:** Главным недостатком является крайне высокая вычислительная сложность, которая растет пропорционально количеству пикселей и объектов. Это делает метод избыточным для задач визуализации в реальном времени при программной реализации на центральном процессоре.

1.2.2 Алгоритм Робертса

Данный алгоритм работает в объектном пространстве и предназначен для удаления граний выпуклых многогранников, нормаль которых направлена в сторону от наблюдателя. Метод вычисляет скалярное произведение вектора взгляда и нормали грани: если значение положительно, грань считается невидимой. **Недостатки:** Алгоритм накладывает жесткие ограничения на входные данные. Он требует идеально корректной топологии моделей со строгим порядком обхода вершин. При наличии ошибок в моделях, таких как неверная ориентация нормалей или незамкнутые объекты, алгоритм работает некорректно, что приводит к появлению артефактов и исчезновению видимых частей объектов.

1.2.3 Алгоритм построчного сканирования с использованием буфера глубины

Данный метод является эффективной комбинацией работы в пространстве изображения и интервальных методов. **Основной принцип работы:** Экран сканируется построчно сверху вниз. Внутри каждой сканирующей строки для каждого активного полигона определяются границы его начала и конца. Для каждого пикселя внутри этого интервала интерполируется значение глубины. Это значение сравнивается с текущим значением в специальном буфере глубины. Если новый пиксель расположен ближе к наблюдателю, он закрашивается, а буфер обновляется новым значением глубины.

Обоснование выбора: Для реализации выбран алгоритм построчного сканирования с использованием буфера глубины. В отличие от алгоритма Робертса, данный метод корректно обрабатывает пересекающиеся объекты и устойчив к ошибкам в топологии моделей. В отличие от трассировки лучей, он легко поддается векторизации с использованием матричных операций, что критически важно для обеспечения высокой производительности при использовании интерпретируемых языков программирования.

1.3 Анализ методов закрашивания

Закрашивание определяет математический способ вычисления цвета каждой точки поверхности с учётом свойств материала и условий освещения. Выбор метода напрямую влияет на реалистичность и скорость рендеринга.

1.3.1 Простое закрашивание

При использовании данного метода для каждой грани вычисляется ровно одна нормаль, как правило, являющаяся перпендикуляром к плоскости грани. Интенсивность освещённости рассчитывается по закону Ламберта как произведение интенсивности источника света на косинус угла падения лучей. Весь полигон целиком закрашивается одним цветом с рассчитанной

интенсивностью, что придает объектам граненый вид. **Преимущества:** Метод требует минимальных вычислительных затрат, так как расчет освещения выполняется один раз на целый полигон, а не для каждого пикселя в отдельности.

1.3.2 Закрашивание Гуро и Фонга

Эти методы основаны на интерполяции параметров освещения между вершинами треугольника. В методе Гуро интерполируется интенсивность цвета, вычисленная в вершинах. В методе Фонга интерполируется сам вектор нормали, и расчет освещенности происходит для каждого пикселя заново. Это обеспечивает визуальную гладкость криволинейных поверхностей.

Недостатки: Оба метода требуют интерполяции дополнительных параметров и выполнения математических операций для каждого пикселя экрана, что значительно увеличивает нагрузку на процессор при программном рендеринге без аппаратного ускорения.

Обоснование выбора: Выбран метод плоского закрашивания. Поскольку рендеринг выполняется на центральном процессоре без использования графического ускорителя, применение более сложных методов приведет к значительному снижению частоты кадров. Плоское закрашивание обеспечивает достаточную наглядность геометрии городских зданий, состоящих преимущественно из плоских поверхностей, при максимальной производительности системы.

1.4 Построение теней

Тени являются важным элементом восприятия глубины, позволяющим зрителю определить взаимное расположение объектов в пространстве.

1.4.1 Проекционные тени

В данном методе трехмерный объект математически проецируется на плоскость поверхности (обычно на землю) с помощью специальной матрицы проекции. Тень рисуется как плоский темный полигон поверх этой поверхности. **Преимущества:** Метод отличается высокой скоростью работы и простотой реализации. **Недостатки:** Тени могут корректно падать только на плоские поверхности. Невозможно реализовать самозатенение объекта или падение тени от одного сложного объекта на другой, если поверхность приема тени не является плоскостью.

1.4.2 Теневые объемы

Это геометрический метод, создающий невидимые объемы пространства, находящиеся в тени за объектами. Для определения того, находится ли пиксель в тени, используется буфер трафарета. **Недостатки:** Метод характеризуется высокой алгоритмической сложностью реализации и создает большую нагрузку на геометрический процессор, особенно при наличии в сцене объектов сложной формы с большим количеством граней.

1.4.3 Метод карт теней

Алгоритм является двухпроходным. На первом этапе сцена рендерится с точки зрения источника света, при этом сохраняется только информация о глубине объектов в специальный буфер. На втором этапе, при отрисовке с камеры, проверяется видимость каждой точки из источника света путем сравнения координат с данными из сохраненного буфера. **Преимущества:** Универсальность метода позволяет теням корректно ложиться на любые объекты независимо от их формы, а также автоматически поддерживает самозатенение.

Обоснование выбора: Выбран метод карт теней. Проекционные тени недостаточно реалистичны для городской среды, где тени могут падать на стены зданий и другие машины. Метод карт теней идеально интегрируется в выбранный конвейер с использованием буфера глубины, позволяя использовать схожие алгоритмы растеризации для обоих проходов визуализации.

1.5 Трёхмерные преобразования

Для управления положением объектов и камеры в пространстве используется аппарат линейной алгебры. Координаты вершин преобразуются умножением на матрицы размерностью четыре на четыре в однородных координатах. Используются мировые преобразования для перемещения и вращения объектов, видовые преобразования для перевода координат в систему отсчета камеры и проекционные преобразования для учета перспективных искажений.

Обоснование: Матричный подход является общепринятым стандартом в компьютерной графике. Он позволяет комбинировать сложные последовательности движений в одну результирующую математическую операцию умножения, что упрощает код и повышает эффективность вычислений.

1.6 Текстурирование

Для повышения визуальной реалистичности сцены без усложнения геометрии используется метод аффинного наложения текстур. Для каждого пикселя растеризуемого треугольника вычисляются соответствующие текстурные координаты путем линейной интерполяции координат вершин. Цвет выбирается из заранее загруженного растрового изображения методом ближайшего соседа.

Обоснование выбора: Аффинное текстурирование является наиболее быстрым способом наложения изображений с точки зрения вычислительных затрат. Метод выборки ближайшего соседа выбран для сохранения четкости текстур при относительно низком разрешении рендера и для исключения размытия, характерного для билинейной фильтрации, что позволяет снизить нагрузку на процессор.

1.7 Организация движения автомобиля и поиск пути

Для реализации автоматического движения автомобиля необходимо выбрать алгоритм поиска кратчайшего пути на графе дорожной сети.

1.7.1 Анализ алгоритма Дейкстры

Алгоритм Дейкстры предназначен для поиска кратчайших путей от одной вершины графа до всех остальных. Он работает с графами, имеющими неотрицательные веса ребер. Алгоритм последовательно выбирает вершину с наименьшей меткой расстояния и релаксирует ребра, исходящие из неё. **Преимущества:** Гарантированно находит кратчайший путь. **Недостатки:** В случае равномерной сетки, где веса всех ребер равны единице, алгоритм Дейкстры выполняет избыточные операции с очередью с приоритетом, что делает его менее эффективным по сравнению с более простыми методами обхода.

1.7.2 Анализ алгоритма A* (A-star)

Алгоритм A* является эвристическим алгоритмом поиска. Он использует оценочную функцию для определения порядка обхода вершин, стараясь двигаться в сторону цели. Это позволяет значительно сократить количество посещаемых вершин по сравнению с алгоритмом Дейкстры. **Преимущества:** Высокая скорость работы при наличии хорошей эвристики. **Недостатки:** Для корректной работы требуется подобрать эвристическую функцию, которая не переоценивает расстояние. В условиях сложного лабиринта дорог с односторонним движением простая эвристика (расстояние по прямой) может работать неэффективно.

1.7.3 Анализ волнового алгоритма

Волновой алгоритм является вариацией поиска в ширину на графе. Он работает путем распространения волны от стартовой точки во все стороны равномерно. Поскольку дорожная сеть представлена в виде сетки с одинаковыми весами ребер, волна гарантированно находит кратчайший путь. **Преимущества:** Идеально подходит для дискретных сеток и лабиринтов. Легко модифицируется для учета запрещенных направлений движения путем проверки матрицы смежности или матрицы направлений.

1.7.4 Выбор алгоритма поиска пути

Для реализации выбран модифицированный волновой алгоритм для ориентированного графа. **Обоснование:** Дорожная сеть в проекте представлена в виде клеточной матрицы, где расстояние между соседними клетками всегда равно единице. В таких условиях волновой алгоритм является оптимальным решением, гарантирующим нахождение кратчайшего пути. Кроме того, использование дополнительной матрицы направлений позволяет легко интегрировать в ал-

горитм правила дорожного движения, запрещая распространение волны против разрешенного потока.

1.8 Выводы по аналитической части

В ходе всестороннего анализа предметной области и существующих алгоритмов компьютерной графики был сформирован следующий стек технологий и методов для реализации курсового проекта:

- **Визуализация:** Выбран алгоритм построчного сканирования с использованием буфера глубины, так как он обеспечивает корректное отображение геометрии и устойчив к ошибкам моделирования.
- **Тени:** Выбран метод карт теней, который обеспечивает построение реалистичных динамических теней от объектов любой формы.
- **Освещение:** Выбрано плоское закрашивание по модели Ламберта, предоставляющее оптимальный баланс между качеством визуализации объема и скоростью работы на центральном процессоре.
- **Логика движения:** Выбран волновой алгоритм на ориентированном графе, позволяющий строить корректные маршруты с учетом правил дорожного движения и односторонних дорог.
- **Реализация:** В качестве инструмента разработки выбран язык Python с использованием библиотеки NumPy. Это позволяет векторизовать матричные операции и компенсировать отсутствие аппаратного ускорения графики.

Сформированный набор методов и алгоритмов позволяет создать функциональную интерактивную систему моделирования движения автомобиля, работающую в режиме реального времени с приемлемой производительностью.

2 Конструкторская часть

В данном разделе представлены требования к разрабатываемому программному обеспечению для трехмерного моделирования движения автомобиля, а также детально описаны алгоритмы и структуры данных, выбранные для реализации системы визуализации.

2.1 Требования к программному обеспечению

Разрабатываемая программа должна предоставлять пользователю графический интерфейс со следующим функционалом:

- загрузка и отображение трехмерной сцены городской среды из конфигурационных файлов;
- управление положением и ориентацией виртуальной камеры в пространстве (перемещение, вращение);
- запуск анимации движения автомобиля по автоматически построенному маршруту;
- отображение текущих параметров производительности (кадров в секунду).

Разработанное ПО должно соответствовать следующим техническим требованиям:

- использование алгоритма Z-буфера для корректного удаления невидимых поверхностей;
- реализация метода карт теней (Shadow Mapping) для построения динамических теней;
- применение модели освещения Ламберта с плоским закрасиванием (Flat Shading);
- поддержка текстурирования объектов сцены;
- расчет маршрута движения с учетом графа дорожной сети и правил дорожного движения.

2.2 Описание структур данных

Для реализации алгоритмов визуализации и моделирования движения используются следующие ключевые структуры данных.

- 1) **Сцена (Scene)** — контейнер верхнего уровня, содержащий списки всех статических (здания, дороги) и динамических (автомобиль) объектов, параметры источника света и объект камеры.
- 2) **Геометрический объект (SceneObject)** включает в себя:
 - массив вершин (dots) — список точек в трехмерном пространстве (x, y, z) ;
 - массив граней (faces) — список полигонов, где каждый полигон задается индексами вершин, цветом и текстурными координатами;
 - вектор положения центра объекта в мировых координатах;
 - матрицу поворота, определяющую ориентацию объекта;
 - вектор направления «вперед» (forward) для корректного движения.

3) **Камера (Camera)** содержит:

- вектор положения в мировом пространстве;
- вектор направления взгляда (target);
- вектор ориентации «вверх» (up);
- матрицы вида (view_matrix) и проекции, пересчитываемые при изменении параметров.

4) **Источник света (ShadowCaster)** включает:

- вектор направления света (для направленного источника);
- параметры интенсивности и цвета;
- собственные матрицы вида и проекции для генерации карты теней;
- буфер глубины (Shadow Map) — двумерный массив для хранения расстояний от источника света до поверхностей.

5) **Дорожная сеть** представлена двумя матрицами:

- матрица проходимости (бинарная: 0 — препятствие, 1 — дорога);
- матрица направлений (битовая маска разрешенных направлений движения из каждой клетки).

2.3 Общий алгоритм построения изображения

Процесс генерации кадра состоит из двух основных проходов: прохода для генерации карты теней и прохода для финальной отрисовки сцены с точки зрения камеры.

На вход алгоритма подаются списки объектов сцены и параметры камеры. На выходе формируется двумерное растровое изображение, которое выводится на экран.

Этап 1: Генерация карты теней (Shadow Pass)

- 1) Устанавливается камера в положение источника света.
- 2) Используется ортогональная проекция для имитации солнечных лучей.
- 3) Производится растеризация всех объектов сцены, но вместо цвета в буфер (Shadow Map) записывается только глубина (z-координата) ближайшего фрагмента.

Этап 2: Финальная отрисовка (Camera Pass)

- 1) Устанавливается камера наблюдателя.
- 2) Инициализируются буфер цвета и Z-буфер (заполняется значением бесконечности).
- 3) Для каждого объекта сцены выполняется конвейер визуализации:
 - трансформация вершин (Model-View-Projection);
 - отсечение невидимых полигонов (отключено для корректной работы с разнородной геометрией);
 - растеризация полигонов методом построчного сканирования (Scanline);
 - для каждого пикселя выполняется тест глубины (Z-test), расчет освещения и наложение текстур.

2.4 Аффинные преобразования

Для размещения объектов в сцене и анимации движения используются матрицы аффинных преобразований размером 4×4 в системе однородных координат.

Матрица поворота вокруг оси Z на угол α (используется для поворота автомобиля):

$$R_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.1)$$

Матрица переноса на вектор (dx, dy, dz) :

$$T(dx, dy, dz) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ dx & dy & dz & 1 \end{pmatrix} \quad (2.2)$$

Итоговое положение вершины V_{new} вычисляется как произведение исходной вершины V на композицию матриц трансформации. В программной реализации для ускорения вычислений используется библиотека NumPy, позволяющая выполнять операции над массивами вершин векторизованно: $V_{new} = V_{array} \cdot M$.

2.5 Приведение к пространству камеры

Для отображения сцены с точки зрения наблюдателя все объекты переводятся в систему координат камеры. Матрица вида (*ViewMatrix*) формируется на основе позиции камеры P , точки направления взгляда T и вектора «вверх» U .

Базис камеры рассчитывается следующим образом:

$$D = \frac{T - P}{\|T - P\|}, \quad R = \frac{D \times U}{\|D \times U\|}, \quad U' = R \times D \quad (2.3)$$

где D — вектор направления, R — вектор «вправо», U' — скорректированный вектор «вверх».

Матрица вида представляет собой комбинацию поворота и переноса, переводящую мир в систему отсчета, где камера находится в начале координат и направлена вдоль оси Z .

2.6 Перспективная проекция

Для создания эффекта перспективы используется матрица перспективной проекции. Она преобразует усеченную пирамиду видимости в куб в нормализованных координатах устройства.

Ключевым этапом является **перспективное деление**: координаты x и y делятся на глу-

бину z .

$$x_{screen} = x \cdot \frac{d}{z + d} \cdot scale + x_{offset} \quad (2.4)$$

$$y_{screen} = y \cdot \frac{d}{z + d} \cdot scale + y_{offset} \quad (2.5)$$

где d — фокусное расстояние, определяющее степень перспективного искажения.

2.7 Алгоритм построчного сканирования с Z-буфером

Для растеризации полигонов (преобразования векторных треугольников в пиксели) используется алгоритм построчного сканирования (Scanline), совмещенный с Z-буфером для удаления невидимых поверхностей.

Схема работы алгоритма:

- 1) Вершины треугольника проецируются на экранную плоскость.
- 2) Определяется ограничивающий прямоугольник полигона по оси Y .
- 3) Происходит итерация по строкам экрана (y) от минимальной до максимальной координаты Y полигона.
- 4) Для каждой строки находятся точки пересечения сканирующей линии с ребрами треугольника (x_{start}, x_{end}).
- 5) Внутри интервала $[x_{start}, x_{end}]$ для каждого пикселя интерполируется глубина z .
- 6) **Z-тест:** Значение z сравнивается с текущим значением в буфере глубины $z_{buffer}[y][x]$.
 - Если $z < z_{buffer}[y][x]$, то пиксель считается видимым. В буфер записывается новое значение z , а в буфер кадра — вычисленный цвет пикселя.
 - Иначе пиксель отбрасывается.

Благодаря использованию библиотеки NumPy, внутренний цикл по X выполняется векторизованно, обрабатывая целую строку пикселей одной операцией, что обеспечивает высокую производительность на интерпретируемом языке Python.

2.8 Расчет освещения и теней

Для закраски пикселей используется модель плоского закрашивания (Flat Shading) с учетом освещенности по закону Ламберта и теней.

- 1) **Нормаль поверхности:** Для каждого треугольника вычисляется вектор нормали N как векторное произведение двух его сторон.
- 2) **Диффузное освещение:** Коэффициент освещенности $I_{diffuse}$ рассчитывается как скалярное произведение нормали N и вектора направления на свет L :

$$I_{diffuse} = |N \cdot L| \quad (2.6)$$

Использование модуля позволяет освещать грани с обеих сторон (актуально для стен без толщины).

3) Расчет тени (Shadow Mapping):

- Координаты текущего пикселя (x, y, z) переводятся в пространство источника света.
- Полученные координаты (u_{light}, v_{light}) используются для выборки глубины из карты теней: $d_{shadow} = \text{ShadowMap}[v_{light}, u_{light}]$.
- Текущая глубина пикселя в пространстве света z_{light} сравнивается с d_{shadow} .
- Если $z_{light} > d_{shadow} + \text{bias}$, то точка находится в тени. Коэффициент освещенности уменьшается до значения фонового света.

Итоговый цвет пикселя вычисляется как произведение цвета текстуры на результирующий коэффициент освещенности.

2.9 Организация поиска пути

Для моделирования движения автомобиля используется **волновой алгоритм** на ориентированном графе.

- 1) Дорожная карта дискретизируется в сетку.
- 2) Каждой ячейке сопоставляется битовая маска разрешенных направлений (Вверх, Вниз, Влево, Вправо).
- 3) Алгоритм начинает распространение волны от стартовой точки. При переходе в соседнюю клетку проверяется битовая маска. Это гарантирует соблюдение правил дорожного движения (например, запрет движения против потока).
- 4) После достижения целевой точки путь восстанавливается обратным ходом от финиша к старту.

Найденный путь преобразуется в список мировых координат, между которыми происходит плавная интерполяция положения и угла поворота автомобиля при анимации.

2.10 Вывод

В разделе разработаны требования к ПО, определены ключевые структуры данных для хранения сцены. Детально описан конвейер визуализации, включающий матричные преобразования, алгоритм построчного сканирования с Z-буфером, метод карт теней и модель освещения Ламберта. Также представлена логика организации движения на основе волнового алгоритма. Выбранные решения оптимизированы для программной реализации и обеспечивают корректное отображение трехмерной сцены в реальном времени.

